reflow: from verifier to agent

Akshaj Shandilya · open-source AI PCB designer (formerly OpenFlux) · repo: github.com/AKSHAJ-SHELL/reflow

> **AI can already draw a circuit board; nothing stops it from handing you one that looks fine and is dead.**

reflow's answer is to put a hand-written **verifier** between the model and the user: rules for real design mistakes, run on every board, each with a plain-English *why* and a concrete *fix*. The verifier is done. This plan covers the next stage, the **harness**: the loop that lets an LLM design a board, get corrected by the verifier, and hand the user something they understand and can manufacture.

5	7	82	0
intent rules working (3 rule files)	fixture boards, incl. a real board clean + with its original bug	tests (33 KiCad-backed, 49 contract)	lines of LLM code so far: the harness is unbuilt

EVIDENCE SO FAR

On the author's real robotics sensor board (13 parts), `reflow check` reports no issues. With the original September mistake restored (a decoupling cap in series with the chip's power pin) it reports **decoupling.series** with the cause and the fix. KiCad's ERC on the same board reports `power_pin_not_driven`: the symptom, with no cause or fix. (On the simpler toy fixture, ERC reports nothing at all.)

Scope of this plan

v1, launch Sun 2026-11-01

- Verifier (done) + provenance and high-speed scope rules
- `reflow design` harness with a ≤3-attempt fix loop
- Part resolver: the AI never picks a part number
- Mandatory learn mode (spot-the-bug) + sign-off
- Simple 2-layer routing + fab outputs

Not in v1

- Simulation / break-it mode
- Project memory vault
- Datasheet profiles, web UI, MCP server
- Multi-sheet, 4+ layers, high-speed or RF routing

01

Why it's hard

1. The dangerous failures are quiet

A board with a series decoupling cap, missing I²C pull-ups or a floating address pin is a perfectly valid netlist. KiCad's ERC reasons about pin *types* (passive, power input), not what a part is *for*, so it can't tell a cap in series from one in parallel. Every quiet failure needs a hand-written rule and a hand-built test board, and a rule that misses a bug still prints "no issues."

2. Models invent part numbers

Ask an LLM for a board and it will confidently write part numbers that don't exist, are out of stock, or don't match the footprint. reflow's rule: the model asks for a *category* ("4.7k resistor, 0402"); a deterministic resolver returns a real part, and any other part number is rejected.

3. Tools grade their own homework

During setup, freerouting reported success on a board where KiCad then found 7 unconnected items. Every stage's output has to be re-checked by something that didn't produce it.

4. KiCad files are enormous; context is not

A schematic file is mostly coordinates and graphics. Feeding it to a model wastes tokens and attention. The harness must send only what the next fix needs: the current board code, the violations, and a one-line memory of what already failed.

5. The loop can thrash

A fix can make things worse, repeat the same mistake, or never compile. The harness needs a hard cap, a rule for which attempt wins, and an honest failure report, not a silent retry.

6. Users stop thinking

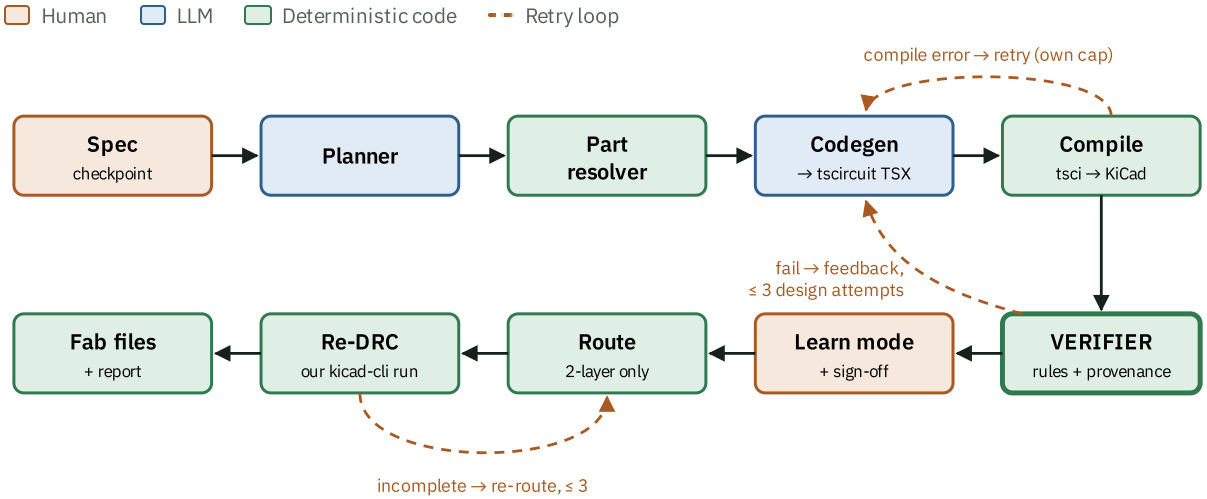
A tool that hands out fab files on demand produces people who can't debug their own boards. Learn mode must make the user engage with a real bug from their own run in about two minutes, without feeling like a quiz.

7. Time

Two weekends plus weeknights to v1, then a hard stop for AMC 12A (Nov 5) and MTFC (from Nov 8). The plan has one go/no-go and a pre-agreed cut list.

02

The method



Every arrow passes one typed contract (reflow/contracts.py): Spec · Plan · ResolvedParts · GeneratedCode · CompileResult · VerifyReport · RetryFeedback · RunResult · LearnRecord · SignOff · RouteResult · FabPackage

01	**Spec checkpoint.** The planner drafts a short spec with open questions; the user answers and confirms. Nothing runs on an unconfirmed spec.	HUMAN
02	**Plan.** Blocks, intended nets, and part *requests* by category. Part numbers allowed only if the user typed them.	LLM
03	**Resolve parts.** Each request becomes a real in-stock part or an honest "unresolved" that stops the run.	CODE
04	**Codegen.** The whole `index.circuit.tsx`, built from a fresh, budgeted prompt every call (no growing chat history).	LLM
05	**Compile.** `tsci` exports KiCad files. Compile errors retry under their own cap and don't consume a design attempt.	CODE
06	**Verify.** The existing rules plus a provenance rule (no unexplained part numbers). Pass → step 8.	CODE
07	**Feed back.** Violations (rule, message, fix hint, refs, nets; never the human-facing *why*) plus a one-line history of failed attempts. Max 3 design attempts; then the best attempt and an honest report.	CODE
08	**Learn + sign off.** The user finds the AI's real first-attempt bug before the fix is shown, then signs off on remaining warnings.	HUMAN
09	**Route + re-check.** 2-layer autoroute, refill zones, re-run DRC ourselves; refuse USB/RF nets; list nets to route by hand if incomplete.	CODE
10	**Output.** Gerbers, drill, pick-and-place, BOM, and the full verification report beside every board.	CODE

Who writes it: the author hand-codes everything that can be *plausibly wrong* (rules, fixtures, the loop, termination, feedback conversion, resolver validity, learn-mode selection). Agents build what fails loudly (CLI, subprocess wrappers, API clients, exporters).

03

How we prove it

Every public claim maps to a measurement anyone can rerun. Numbers get published as they come out, including bad ones.

MEASUREMENT	HOW	CLAIM IT SUPPORTS
Bugs caught on known-bad boards	`reflow check` on every bad fixture (one planted bug each, incl. a real board with its real bug); count exact rule-ID matches	"reflow catches these specific mistakes"
False alarms on known-good boards	Good fixtures, incl. the real AVBotz board, must report zero violations	"It doesn't cry wolf on real boards"
KiCad ERC side by side	Run `kicad-cli sch erc` on the same bad boards; record exactly what it says	"ERC misses or only hints at these" (worded per board, never overstated)
First-pass vs post-retry pass rate	Eval set of ≥5 board prompts × 3 runs; fraction verifier-clean on attempt 1 vs within 3 attempts	"The fix loop works" (or how much)
Compile success + compile retries	Per run: compile tries per attempt, failed-compile runs	"The model writes valid tscircuit" (or doesn't)
Hallucinated-part rate	Part numbers in raw model output not from the resolver or the user, before the gate; and in final outputs, after it	"The model invents parts X% of the time; reflow ships 0"
Routing completeness	% of eval boards fully routed and DRC-clean on our re-run; count router-vs-KiCad disagreements	"Simple boards route; we say when they don't"
Tokens per run	Median and p90 input/output tokens from `run.json`	"The harness is context-efficient"
Learn-mode engagement (post-launch)	Share of testers who name the right part before the reveal; time spent	"Users engage, not click through"

Already measured (2026-10-17)

BOARD	KICAD ERC	REFLOW CHECK
`avbotz_good` (real, 13 parts)	not recorded	no issues
`avbotz_series_cap` (real, original bug)	1 error: `power_pin_not_driven` on U1 pin 8	**decoupling.series**, with cause + fix
`bad_series_cap` (toy)	0 errors, 0 warnings	**decoupling.series**
`bad_floating_addr` (toy, no-connect flag on A0)	0 errors, 0 warnings	**address.floating**

Test suite: 82 tests across 7 fixtures. The eval set (E-02) and eval runner (E-01) don't exist yet; no harness numbers are claimed until they do.

04

Build plan to v1

Human (hand-coded) · Agent · Launch / demo · First to cut · ◆ go/no-go

TASK		SA 17	SU 18	M 19	T 20	W 21	TH 22	F 23	SA 24	SU 25	M 26	T 27	W 28	TH 29	F 30	SA 31	SU 1
Launch `reflow check` (README, post)	D-01																
Finish rename, untrack scratch	Z-01/02																
Harness decisions (Q3–Q6, D-1…D-6)	H-00																
Model adapters + config	M-01/02																
Compile wrapper + error parser	K-01/02																
Harness skeleton → loop → termination	H-01…04																
Prompts (agent draft, human approves)	H-08																
Parts catalog client	P-01/03																
Resolver validity + ranking	P-02																
Eval prompt set (≥5 boards)	E-02																
◆ **Go/no-go** (end-to-end on ≥3/5)																	
Provenance + high-speed scope rules	R-03/04/05																
Router wrapper + re-DRC	X-01/04																
freerouting fallback	X-02																
Learn mode logic (human) + screens	L-01…03																
Fab export + report.json/md	O-01/02																
report.html	O-03																
Eval run + numbers	E-01																
Demo recording + v1 launch	D-02																

Weekend columns shaded. Weekday bars assume ~1–2 h on weeknights. Hard stop after: AMC 12A Thu Nov 5, MTFC from Nov 8, AMC 12B Nov 13.

Go/no-go, Sun Oct 25 evening

`reflow design "MCP9808 breakout, I2C header, 3.3V"` and the eval prompts: a verifier-clean compiled board within 3 attempts on ≥3 of 5 prompts, with zero part numbers outside the resolver.

No-go → routing moves to v1.1; launch "design + verify; you route" on Nov 1.

Cut order (first cut first)

1. freerouting fallback (keep one router)
2. `report.html` (keep `report.md`)
3. make-the-call interaction (keep spot-the-bug)
4. routing entirely → v1.1

Never cut: verifier, provenance gate, spot-the-bug, honest failure report.

05

Risks

RISK	FALLBACK
The model writes TSX that compiles but wires the board wrong in ways no rule covers	Eval prompts carry hand-written expected nets; narrow v1 to a few board families (sensor breakout, LED, regulator) and say so
tscircuit's KiCad export splits power nets (known issue #595) or adds ERC noise	Verify from circuit-json nets as a cross-check; report export artifacts separately, never drop them
`tsci` is a Node/Bun dependency inside a Python tool	Pin the version; a `reflow doctor` check; document in SETUP
Parts backend (jlcsearch) changes or goes down	Local SQLite snapshot (P-03); unresolved parts stop the run honestly, never guessed
KiCad 11 removes SWIG `pcbnew` DSN/SES	Pin KiCad 10; tscircuit's router is the default, freerouting only a fallback
freerouting (GPL), easyeda2kicad and AlphaCodium (AGPL) licensing	Separate processes only; never imported or vendored; AlphaCodium used as a cited pattern only
Fix loop thrashes or never compiles; token cost spikes	Hard caps (3 design attempts, separate compile cap), early stop on repeated identical feedback (if chosen), per-run token totals logged
Model output slips a part number past the guard	Provenance rule as a final gate + contract test; measured hallucinated-part rate published
Learn mode reads as a quiz and users click through	Only real bugs from the user's own run; 1–3 interactions; wrong answers teach, never block; measure engagement after launch
Overclaiming in public (e.g. "ERC is silent")	Integrity rules: claims per board, from recorded runs; comparisons use published prompts and outputs
Trademark / framing against a funded competitor	Name is reflow; "an open-source alternative to tools like Flux"; never "killing Flux"
Schedule collides with AMC/MTFC	Go/no-go on Oct 25, pre-agreed cut list; nothing scheduled after Nov 1
Agents edit hand-owned code or weaken tests	AGENTS.md golden rules; OWNER column on every task; contract tests freeze schemas; review `git diff --stat`

06

Decisions for me

Options only. Nothing here is picked; accepted choices go into DECISIONS.md.

D-1 · LLM provider / model

- a **Anthropic Claude (Sonnet tier).** Strong at code, prompt caching, native structured outputs. Paid per run.
- b **OpenAI (GPT tier).** Comparable; native structured outputs; tscircuit's own prompt benchmarks are wired to OpenAI.
- c **Local via Ollama** (incl. Ornith later). Free and private; likely weaker at TSX. Good as a comparison arm.

D-2 · Framework vs hand-rolled

- a **Hand-rolled loop + 2–3 small adapters.** ~150–200 lines you fully understand; you maintain adapters.
- b **Hand-rolled loop + LiteLLM** for model I/O. 100+ providers free; large, fast-moving dependency.
- c **PydanticAI.** Typed outputs and validation retries built in; its abstraction hides the prompts you want to own.

D-3 · Feedback format (Violation → model)

- a **Compact text block** per violation. Fewest tokens; least structure.
- b **RetryFeedback as JSON.** Unambiguous, matches the contract; a few more tokens.
- c **JSON + inline comments in the TSX** at offending refs. Points at the exact spot; needs ref→line mapping.

D-4 · Learn-mode interaction

- a **Spot-the-bug, multiple choice.** Fast and easy to grade; guessable.
- b **Spot-the-bug, free text** graded by matching refs. Makes people think; fuzzier grading.
- c **Make-the-call first** (pick a value, instant re-verify). Feels interactive; needs rules with tunable values.

D-5 · UI form for v1

- a **CLI + generated report.html.** Visual for the demo; no server.
- b **CLI only.** Least work; weakest demo next to polished tools.
- c **Local web app.** Best experience; most work (currently v2).

D-6 · Part API

- a **jlcsearch public JSON API** (tscircuit). Zero setup; depends on a third-party service.
- b **Local SQLite snapshot** (jlcsearch db, <2 GB, or jlcparts). Offline and reproducible; big download, refresh needed.
- c **Bring-your-own-key distributor APIs** (Mouser, DigiKey). Authoritative stock; setup friction; no shared keys allowed.

Also open (harness, docs/06): Q3 which attempt the user gets when fixes make things worse · Q4 crash safety / resume · Q5 stop early on repeated identical feedback · Q6 where the part-number check runs · the compile-retry cap value.

07

Sources

All opened on 2026-10-17 (via web fetch). GitHub's commit pages were not readable by the fetch tool, so "latest" dates are release/PyPI dates.

ITEM	URL	LICENSE · LATEST SEEN
tscircuit / tsci CLI	github.com/tscircuit/tscircuit, github.com/tscircuit/cli	MIT · very active (date unverified)
tscircuit LLM docs / agent skill	docs.tscircuit.com/llms.txt, github.com/tscircuit/skill	MIT
tscircuit prompt-benchmarks	github.com/tscircuit/prompt-benchmarks	MIT
jlcsearch	github.com/tscircuit/jlcsearch, jlcsearch.tscircuit.com	MIT
jlcparts	github.com/yaqwsx/jlcparts	MIT (code) · data license not stated
KiCad 10 kicad-cli	docs.kicad.org/10.0/en/cli/cli.html	— · no DSN/SES in kicad-cli
kicad-python (IPC API)	gitlab.com/kicad/code/kicad-python	MIT · 0.8.0, 2026-08-30
freerouting	github.com/freerouting/freerouting	GPL-3.0 · v2.4.1 (Sep, year inferred)
InteractiveHtmlBom · KiCanvas	github.com/openscopeproject/InteractiveHtmlBom · github.com/theacodes/kicanvas	MIT · v2.12.0 · KiCanvas "early alpha"
easyeda2kicad	github.com/uPesy/easyeda2kicad.py	AGPL-3.0 · 1.0.1, 2026-04-06
atopile · SKiDL · circuit-synth	github.com/atopile/atopile · github.com/devbisme/skidl · github.com/circuit-synth/circuit-synth	MIT · 0.15.9 (2026-09-12) · 3.0.0 (2026-10-16) · v0.12.1
mini-swe-agent	github.com/SWE-agent/mini-swe-agent	MIT · v2.4.6, 2026-07-23
Building effective agents	anthropic.com/engineering/building-effective-agents	article, 2024-12-19
OpenAI Agents SDK · PydanticAI · LangGraph · smolagents	github.com/openai/openai-agents-python · github.com/pydantic/pydantic-ai · github.com/langchain-ai/langgraph · github.com/huggingface/smolagents	MIT · MIT · MIT · Apache-2.0
Claude Agent SDK (Python)	github.com/anthropics/claude-agent-sdk-python	MIT (+ Anthropic terms) · 0.2.168
LiteLLM	github.com/BerriAI/litellm	MIT (enterprise/ separate) · 1.105.1
AlphaCodium · Aider	github.com/Codium-ai/AlphaCodium · github.com/Aider-AI/aider	AGPL-3.0 · Apache-2.0
Pydantic · Instructor · jsonschema	github.com/pydantic/pydantic · github.com/567-labs/instructor · github.com/python-jsonschema/jsonschema	MIT · MIT · MIT
Structured outputs docs	platform.claude.com/docs/en/build-with-claude/structured-outputs · platform.openai.com/docs/guides/structured-outputs	—
MCP spec + Python SDK	modelcontextprotocol.io/specification/latest · github.com/modelcontextprotocol/python-sdk	MIT · mcp 2.3.0
Flux MCP server (spec reference only)	docs.flux.ai/reference/flux-mcp-server	capabilities listed; tool names not published
KiCad MCP servers	github.com/mixelpixx/KiCAD-MCP-Server · github.com/lamaalrajih/kicad-mcp	MIT · MIT
JLCPCB / LCSC / Mouser / DigiKey / Nexar APIs	api.jlcpcb.com · lcsc.com/agent · Mouser, DigiKey v4, Nexar developer portals	application or per-user keys; no shared keys

Not verified

- Latest-commit dates for every GitHub repo (release dates used instead)
- circuit-json license; tscircuit repo dates
- A non-interactive mode for `tsci import`
- Whether tscircuit can import an SES back into circuit-json
- KiCad 10 netlist `--format` list
- Release years for freerouting v2.4.1 and iBOM v2.12.0
- kicad-jlcpcb-tools database file names/format
- Nexar free-tier size (sources disagree: 100 vs 1,000)
- Terms of Mouser/DigiKey/Octopart APIs; jlcparts data redistribution rights
- Flux MCP exact tool names
- Token estimates in docs/06 (to be measured)
- Claude Agent SDK support for non-Anthropic models

Companion docs in the repo: docs/00–08, TASKS.md, AGENTS.md, DECISIONS.md. Example LCSC part in docs/examples is illustrative.